

Solidity Guardian: Validating Developer Intentions in Smart Contracts via Intent Annotations to Mitigate Numeric Logical Vulnerability

Inseong Jeon¹, Sundeuk Kim¹, Hyunwoo Kim¹, Hoh Peter In^{1*}

^{1*}Department of Computer Science, Korea University, 145, Anam-ro,
Seonbuk-gu, 02841, Seoul, Republic of Korea.

*Corresponding author(s). E-mail(s): hoh_in@korea.ac.kr;
Contributing authors: iwwyou@korea.ac.kr; sd_kim@korea.ac.kr;
khw0809@korea.ac.kr;

Abstract

In smart contract development, arithmetic issues such as integer division truncation, incorrect operation order, and scaling mismatches can silently violate the developer’s intended numeric relationships—yet still result in successful transactions. These *numeric logical vulnerabilities* typically evade runtime exceptions, static analysis rules, and dynamic testing, often leading to significant financial losses after deployment. This paper introduces **Solidity Guardian**, a lightweight tool that enables developers to declare intended numeric invariants directly in code and receive immediate feedback on compliance. By leveraging an *Intent DSL*—structured around `@Pre`, `@During`, and `@Post` phases—and combining it with SolQDebug’s interval-based abstract interpretation engine, Solidity Guardian continuously tracks variable ranges from function entry to exit. The system performs incremental parsing and selective control-flow analysis, allowing it to deliver deterministic **True/False/Unknown** verdicts within milliseconds (average 30 ms for functions up to 500 lines), seamlessly integrating with the development workflow. Our results demonstrate that this “intent annotation + deterministic checking” approach effectively prevents subtle numeric logic errors during development, proposing a new paradigm for mechanically verifying developer intent in smart contract programming.

Keywords: Smart Contract Development, Solidity, Numeric Vulnerabilities, Intent Annotations, Abstract Interpretation, Developer Tools

1 Introduction

Smart contracts handle assets directly by performing arithmetic operations on integer values such as token balances, fees, and percentages. While Solidity 0.8 and later versions enforce automatic overflow checks, logical numeric errors—such as incorrect operation order, integer division truncation, and scaling mismatches—remain elusive and are not easily detected by conventional means.

Although large language models (LLMs) have recently been introduced to support tasks such as code completion, refactoring, and bug explanation, they often struggle with precise value inference and boundary condition analysis due to their generative nature. Crucially, any financial loss resulting from LLM-generated code ultimately becomes the developer’s responsibility, underscoring the need for rigorous validation of numeric invariants during development. A notable real-world example is the 2023 Sperax DAO incident. A single-line bug in the `_balanceOf` function caused account balances to be massively overstated. Since the transaction executed successfully, monitoring systems failed to detect the issue until after approximately \$300,000 worth of tokens had been exploited.

Existing tools often miss such vulnerabilities. Static analyzers excel at detecting predefined patterns (e.g., unsafe `tx.origin` use or unchecked external calls) but are not designed to infer specific numeric intentions like “distribute 10% of reward.” Fuzzers and symbolic execution engines may fail to hit critical boundary conditions, especially when confronted with path explosion in complex mappings, loops, or modifiers. Comment-code inconsistency (CCI) detectors provide probabilistic assessments based on weak correlations between natural language comments and code but cannot deliver definitive True/False verification. Formal verification offers precision but demands exhaustive specification efforts, rendering it impractical for most real-world projects. Thus, there is a pressing need for a lightweight mechanism that allows developers to express numeric intent alongside code and instantly verify its correctness.

While our Intent DSL is designed with domain-agnostic primitives that can express various logical properties, we focus exclusively on numeric vulnerabilities in this work for three reasons. First, numeric variables dominate smart contract logic: our analysis of the DappSCAN dataset reveals that over 82% of non-trivial functions manipulate numeric types, primarily in financial operations such as token transfers, fee distributions, and reward calculations. Second, numeric errors have direct and measurable financial impact, providing clear ground truth for evaluation—the Sperax DAO incident alone resulted in \$300,000 in losses. Third, and most importantly, CHEN *et al.*’s NumScout dataset (Chen et al., 2025) provides the only benchmark with explicitly documented intent-behavior mismatches, enabling rigorous evaluation of intent validation approaches. Other logical properties, such as access control or state consistency, lack similar datasets with ground-truth developer intentions, making systematic evaluation infeasible at present.

To fill this gap, we propose Solidity Guardian, a tool that enables developers to declare intent with a single-line annotation adjacent to the code. Our domain-specific language (DSL) adopts a three-phase model—`@Pre`, `@During`, and `@Post`—to specify initial conditions, runtime constraints, and postconditions, respectively. Solidity

Guardian leverages SolQDebug’s interval-based abstract interpretation engine to compute variable ranges in the background and checks them against the declared intent. The system performs partial re-analysis only on the changed code fragments, returning compliance results—**Satisfied**, **Violated**, or **Unknown**—within an average of 30 milliseconds for functions up to 500 lines. As a result, developers receive real-time feedback via visual cues directly within the IDE, without disrupting their workflow.

This paper makes the following contributions:

- **Intent DSL and Formal Semantics:** We define a lightweight annotation language based on three execution phases and formalize its semantics in terms of interval state transitions.
- **Millisecond-Scale Verification Engine:** By reusing SolQDebug’s dynamic control-flow graph (CFG) and interval analysis, we achieve millisecond-level feedback through partial recomputation.
- **Large-Scale Empirical Evaluation:** Our evaluation on 1,023 DappSCAN functions and the NumScout dataset (95 labeled contracts, 11 with Solidity 0.8+ vulnerabilities) demonstrates Guardian’s effectiveness in detecting numeric logic errors that evade existing tools.
- **Practical Use Cases:** We analyze scenarios involving tokenomics (e.g., fee distributions, reward sharing, burn limits), staking contracts, and AMM exchange rates to illustrate how Guardian provides immediate and actionable feedback.

The rest of this paper is organized as follows. We first review Solidity’s numeric types and real-world examples of numeric logic flaws, followed by an analysis of why existing tools fail to detect them. We then introduce the Intent DSL and its formal semantics, followed by the design of Guardian’s verification engine based on partial re-analysis. Subsequently, we present our evaluation results on responsiveness and detection effectiveness, discuss practical applications, limitations, and future directions, and conclude with a comparison to related work.

2 Background

2.1 Solidity and Its Numeric Types

Solidity is a statically typed, contract-oriented programming language designed for the Ethereum Virtual Machine (EVM) ([Solidity Documentation, 2025](#); [Wohrer and Zdun, 2018](#)). Its syntax borrows elements from C++, JavaScript, and Python, and it supports modular development through contracts composed of variables and functions ([Wohrer and Zdun, 2018](#); [Language Influences, 2025](#)). As a Turing-complete language, Solidity enables the implementation of complex decentralized applications with persistent on-chain state.

Solidity supports a range of built-in data types ([Types, 2025](#)):

- **Booleans** — logical `true` or `false` values.
- **Integers** — signed (`intN`) and unsigned (`uintN`) integer types with widths from 8 to 256 bits.

- **Fixed-point numbers** — declared but currently unsupported in arithmetic operations.
- **Address** — 20-byte externally owned accounts (EOAs) or contract addresses.
- **Fixed-size byte arrays** (`bytesN`) — statically sized byte sequences.
- **Strings and dynamic bytes** — variable-length UTF-8 strings or byte sequences.
- **Enums** — user-defined enumerated types.
- **User-defined value types** — new type aliases with stricter invariants based on elementary types.

Variables of these types can be declared in three storage categories ([Units and Globally Available Variables, 2025](#); [Structure of a Contract, 2025](#)):

- **Global variables** — predefined variables and functions that expose blockchain metadata or serve as utility features (e.g., `msg`, `block`, `tx`).
- **State variables** — persistent contract storage retained across transactions.
- **Local variables** — ephemeral data residing in a function’s execution context.

Among these, numeric variables play a dominant role in smart contract logic. To quantify this, we analyzed a DappSCAN-Source dataset ([Zheng et al., 2024](#)) comprising contracts written in Solidity version 0.8.0 or later. From 3,345 available contracts, we randomly selected approximately 10% of contracts from each size bracket (1–10 KB, 11–20 KB, and over 20 KB), yielding 103 Solidity files containing 128 contracts and 1,524 functions in total. We excluded 501 trivial functions, which consist solely of assignments, simple returns, or void function calls, as they lack substantive logic relevant to intent specification.

Table.1 summarizes our findings. Among the remaining 1,023 functions, over 82% use numeric variables, with `uint` appearing in more than 78% of cases. The prevalence of unsigned integers reflects the dominant use of numeric logic in token-related operations such as `mint`, `transfer`, `withdraw`, `burn`, `stake`, and `reward`. These functions typically enforce non-negative balances, making `uint` the preferred type. This empirical evidence highlights that numeric types are pervasive in Solidity contracts, underscoring the importance of carefully handling arithmetic operations during development to prevent critical logic errors.

2.2 Numeric Logical Vulnerabilities in Smart Contracts

In blockchain systems, even transactions that complete successfully at runtime may harbor severe *numeric logical vulnerabilities* if they violate the developer’s intended logic due to subtle errors such as incorrect operation order, integer division truncation, or faulty branching conditions. Unlike runtime exceptions, these logic errors do not trigger reverts or explicit failures. Consequently, they can bypass both conventional testing and on-chain monitoring, becoming permanently recorded on the blockchain.

A notable real-world example is the *Sperax DAO incident* in February 2023. The root cause was a one-line logic error in the `_balanceOf` function (Figure ??), which mistakenly returned the raw `credits` variable instead of performing the intended normalization `credits / creditPerToken`. This seemingly minor mistake led to an inflated account balance that exceeded the correct value by several hundred times.

```

1 // ..Code for other functions are omitted
2 function _ensureRebasingMigration(address _account) internal {
3     if (nonRebasingCreditsPerToken[_account] == 0) {
4         nonRebasingCreditsPerToken[_account] = 1;
5         if (_creditBalances[_account] != 0) {
6             uint256 bal = _balanceOf(_account);
7             nonRebasingSupply = nonRebasingSupply.add(bal);
8             _creditBalances[_account] = bal;
9         }
10    }
11 }
12 function _balanceOf(address _account) private view returns (uint256) {
13     uint256 credits = _creditBalances[_account];
14     if (credits > 0) {
15         if (nonRebasingCreditsPerToken[_account] > 0) {
16             // The code should have been
17             // credits/creditPerToken
18             return credits;
19         }
20         return credits.divPrecisely(rebasingCreditsPerToken);
21     }
22     return 0;
23 }
24
25 // ..Code for other functions are omitted

```

Fig. 1: Motivation Example (BlockSecTeam, 2023)

The overestimated balance was subsequently aggregated into the total supply via the `_ensureRebasingMigration` function. Because all transactions executed successfully without exceptions, on-chain monitors failed to detect the anomaly. By the time the exploit was noticed, attackers had already drained tokens worth approximately \$300,000 under the guise of legitimate withdrawals.

Furthermore, CHEN *et al.* (Chen et al., 2025) conducted a systematic analysis of 1,199 audit reports from the DappSCAN dataset (Zheng et al., 2024), identifying five distinct categories of novel numeric logical vulnerabilities:

- **Div-In-Path:** Integer division within conditional expressions distorts control flow. A condition such as `if (msg.value / 1 ether > 3)` wrongly evaluates to false for 3.9 ETH, failing the branch. Eleven real-world attack cases were documented
- **Operator-Order Error:** Arithmetic operations applied in the wrong order, causing premature integer truncation. For example, `reward / 100 * 10` always evaluates to zero when `reward < 100`.
- **Minor Retention:** Rounding remainders (e.g., fees or interest) become permanently locked in the contract. In one case, successive remainder accumulation in a distribution loop resulted in 90 ETH remaining locked over three months.
- **Exchange Problem:** Scaling factor mismatches cause exchange rate deviations by up to 10^{-18} . Four DeFi AMM contracts were observed to exhibit slippage losses exceeding 0.3%.

- **Precision-Loss Trend:** Repeated multiplication and division operations amplify rounding errors over time, potentially driving balances negative in contract end states. This vulnerability forced emergency upgrades in two staking pool contracts.

Since Solidity 0.8 introduced automatic integer overflow checks with revert semantics, traditional *integer overflow* vulnerabilities have been largely mitigated. However, the aforementioned logical numeric errors remain undetectable by runtime safeguards, as they manifest only in semantically incorrect—but syntactically valid—execution outcomes. Therefore, unless developers explicitly verify numeric invariants on critical quantities such as amounts, ratios, or balances during development, these vulnerabilities are virtually impossible to detect using conventional methods.

2.3 Limitations of Existing Detection Approaches

Smart contract analysis tools have advanced across various categories, including static analysis, dynamic analysis, comment-code inconsistency (CCI) detection, and formal verification. However, most existing tools struggle to deterministically detect numeric logic errors—cases where the execution result violates the developer’s intent despite syntactically correct code. Consider the following simple yet critical example:

```
1 // Intended: 10% of reward
2 // Actual: 0 wei (when reward < 100)
3 devReward = reward.div(100).mul(10);
```

If `reward` is between 1 and 99, the integer division `reward / 100` rounds down to zero, and the multiplication yields zero regardless of the intended 10% computation. The compiler emits no warnings, nor does the overflow/underflow checker flag this issue. Why does this happen?

(1) Static Analysis (Pattern- and Rule-Based)

Tools like SLITHER and MYTHRIL focus on detecting fixed-form defects such as missing SAFEMATH checks, common reentrancy patterns, or misuse of `tx.origin`. However, errors related to arithmetic order, rounding behavior, or control-flow distortion—which require comparing intended versus actual numeric values—are difficult to encode as static patterns or rules. Consequently, expressions like the example above typically pass static analysis as “clean.”

(2) Dynamic Analysis, Fuzzing, and Concolic Execution

Tools like ECHIDNA and SFUZZ explore execution paths through randomized or symbolic inputs. Nevertheless, discovering a critical boundary input (e.g., `reward = 37`) that triggers a logic violation is inherently difficult. Moreover, functions with complex mappings, loops, or modifiers exacerbate state-space explosion, causing tools to hit depth limits and miss violations.

(3) Comment-Code Inconsistency (CCI) Detection with Natural Language Processing

Recent LLM- or embedding-based approaches attempt to align natural language comments (e.g., “distribute 10% fee”) with code semantics. However, these models struggle

to *prove* a precise numeric relationship—such as “exactly 10%”—and instead return only probabilistic mismatch scores. If comments are absent or ambiguous, this gap widens further.

(4) *Formal Verification (e.g., KEVM, VeriSmart)*

Formal verification frameworks require developers to write explicit invariants or state-machine specifications. In practice, the steep learning curve and high annotation cost limit their adoption in real-world projects. Any code segments lacking formal specifications remain unanalyzed, creating blind spots.

As a result, numeric logic bugs like the example above are permanently recorded on-chain as **success** transactions. Developers often only discover the problem after user losses occur. Two critical gaps explain why existing tools fail to catch these issues:

- **Lack of Intent Annotations:** Source code alone provides no way to infer that, for example, a value should represent exactly 10% of another.
- **Lack of Deterministic Checking:** Even when intent is described—through comments or annotations—existing tools cannot deterministically decide the correctness of a single arithmetic expression at the line level.

To fill this gap, we propose SOLIDITY GUARDIAN, a framework that allows developers to declare their intended numeric logic using a single-line domain-specific language (DSL). The system then applies interval-based abstract interpretation to compute and compare actual outcomes within milliseconds, providing immediate **True/False** feedback. This approach offers deterministic verification at sub-wei precision, addressing a critical need that previous tools have left unfilled.

2.4 SolQDebug: Foundation for Guardian

SolQDebug is a source-level Solidity debugger designed to enable interactive abstract interpretation during code authoring. Rather than relying on traditional deployment and transaction-based testing workflows, SolQDebug performs interval-based static analysis on incomplete code fragments and reports symbolic value ranges directly in the Solidity editor. This millisecond-scale feedback loop allows developers to inspect how symbolic inputs propagate through variables and control structures, without compiling or deploying contracts.

At the core of SolQDebug is an extended Solidity grammar that supports inline annotations for symbolic inputs. Developers declare ranges for global, state, and local variables using one-line directives (e.g., `// @StateVar x = [0, 100]`). The system incrementally constructs a dynamic control-flow graph (CFG) from live edits and executes abstract interpretation over a fixed-point lattice of interval values. Each edit updates only the affected region, enabling fast recomputation. A test-case isolation mechanism ensures that symbolic inputs do not interfere across runs.

In the context of this work, we extend SolQDebug’s analysis core as the verification backend for *Solidity Guardian*. Specifically, the interval domain interpreter, dynamic CFG builder, and per-statement memory model are adopted as-is. However, whereas SolQDebug was designed to compute “what values a variable can take,” Guardian checks “whether the observed behavior conforms to the developer’s intent.”

To support this shift in goal, we augment the interpreter to compare computed intervals against developer-declared postconditions and invariants. These logical intent specifications, written in a new annotation language (Section 4.1), allow Guardian to verify correctness properties such as value preservation, expected directional change, or relational constraints across entry, assignment, and exit. The reuse of SolQDebug’s infrastructure ensures that verification is lightweight, precise, and responsive—consistent with Guardian’s goal of bridging the gap between program logic and developer intent.

2.5 Proposed Work: Solidity Guardian

To address the two critical gaps identified in the previous section—(1) the inability to infer a developer’s numeric intent from code alone and (2) the lack of a mechanism to deterministically check whether a single-line arithmetic expression satisfies that intent—we propose **Solidity Guardian**. The core idea is straightforward: Developers declare numeric invariants, such as “this value must be 10%” or “`balance` should be non-negative after the loop,” using a single-line *Intent DSL* placed directly alongside the code. Each time the code is edited, Solidity Guardian computes the actual range of the relevant variables within milliseconds and compares it against the declared intent. The outcome is deterministically categorized into one of three states—**True**, **False**, or **Unknown**—and immediately displayed in the editor using intuitive visual cues.

Guardian’s Solution

Solidity Guardian addresses both gaps through:

- **Input Seeding:** Developers specify initial ranges to explore critical scenarios.

```
1 // @GlobalVar msg.value = 1
2 // @StateVar balances[attacker] = [1000, 2000]
3 // @LocalVar reward = [1, 99]
```

- **Intent Specifications:** Declare expected numeric relationships.

```
1 // @During devReward(Assign == Current * 10 / 100)
2 // @Post returnExpression == reward * 10 / 100
```

Deterministic Checking Procedure

Upon saving the file or confirming a line of code, Solidity Guardian re-parses only the modified code fragment. It leverages SolQDebug’s verified dynamic control flow graph (CFG) and interval propagation engine to compute the actual value intervals of variables.

$$\textbf{Satisfied} \iff I_{\text{actual}} \subseteq I_{\text{intent}}$$

$$\textbf{Violated} \iff I_{\text{actual}} \cap I_{\text{intent}} = \emptyset$$

$$\textbf{Uncertain} \iff I_{\text{actual}} \not\subseteq I_{\text{intent}} \wedge I_{\text{actual}} \cap I_{\text{intent}} \neq \emptyset$$

If neither satisfied nor violated, the system conservatively reports **Unknown**. The entire verification process is implemented in a Python backend, consistently completing within an average of 30 milliseconds even for functions around 500 lines long.

Key Advantages

- **Developer-Centric Intent Declarations:** Numeric invariants can be expressed with a single line of comment-like syntax, reducing the learning curve.
- **Millisecond-Level Responsiveness:** Partial recalculations allow feedback with minimal latency, comparable to keystroke delays.
- **Deterministic Results:** By providing clear `True/False` outcomes, the tool reliably detects even sub-wei deviations.

In the following sections, we present the formal *Intent Model* used by Solidity Guardian and detail the verification procedure that determines compliance between declared intent and actual execution outcomes.

3 The Design of Solidity Guardian

3.1 System Architecture

Solidity Guardian extends SolQDebug’s interval-based abstract interpretation infrastructure to verify developer-declared numeric intents. Figure ?? illustrates the system architecture and workflow.

Core Components

Guardian comprises five main components:

- **Intent Parser:** Processes Guardian DSL annotations (`@GlobalVar`, `@StateVar`, `@LocalVar`, `@During`, `@Post`) using an ANTLR4 grammar (Solidity.g4) extended with intent-specific productions. The parser constructs abstract syntax trees for intent expressions, distinguishing them from regular Solidity code.
- **Dynamic CFG Builder:** Incrementally constructs and updates a control-flow graph from live code edits. Inherited from SolQDebug, this component performs partial re-analysis on changed code regions only, enabling millisecond-scale responsiveness. Each CFG node maintains interval states for variables at that program point.
- **Debug Initializer:** Seeds the initial abstract state from `@GlobalVar`, `@StateVar`, and `@LocalVar` annotations before analysis. This allows developers to explore specific attack scenarios (e.g., setting `msg.value = 1` to test indivisible amount bugs) rather than relying solely on symbolic ranges.
- **Expression Evaluator:** Computes interval values for Guardian DSL expressions, including numeric helpers (`PercentOf`, `ceil`, `floor`), temporal snapshots (`Before/After`, `Entry/Exit`), and return value references. Reuses SolQDebug’s evaluator for standard Solidity expressions, adding Guardian-specific context handlers.
- **Verification Engine:** Implements three-valued logic for intent checking. For each `@During` or `@Post` specification, the engine retrieves the relevant interval states, evaluates both sides of the comparison, and determines **Satisfied** (property definitely holds), **Violated** (property definitely fails), or **Unknown** (intervals overlap) with a confidence score. Section ?? details the interval comparison algorithm.

Workflow

Guardian operates in a continuous feedback loop:

1. **Edit**: Developer writes code and annotations in the IDE.
2. **Parse**: Intent Parser extracts `@GlobalVar/@StateVar/@LocalVar` seeds and `@During/@Post` specifications.
3. **Analyze**: Dynamic CFG Builder updates the control-flow graph, Debug Initializer applies seeds, and the Evaluator propagates intervals through the CFG.
4. **Verify**: Verification Engine checks each intent specification against computed intervals.
5. **Report**: Results (`Satisfied/Violated/Unknown` + confidence scores) are displayed inline in the editor within 30ms.

By reusing SolQDebug’s verified interval propagation logic and CFG infrastructure, Guardian inherits its soundness guarantees while adding intent-checking capabilities.

3.2 Intent Model

Goal.

SOLIDITY GUARDIAN provides a light-weight, single-line annotation language that lets developers state—next to the code—*what should hold*, not how to prove it. Each annotation is checked incrementally after every edit against an interval abstract state, producing a deterministic `True/False/Unknown` in a few milliseconds, plus a numeric *confidence* and, when useful, a *diagnostic interval* explaining where the uncertainty comes from.

Top-level shape. An annotated file is a bag of *intent directives* partitioned into three phases:

`@Pre @During @Post`

They may appear in any textual order but are interpreted in the semantic order: *seed injection* $\rightarrow$ *mid-execution checks* $\rightarrow$ *exit-time checks*. The entry non-terminal is `intentUnit`.

Listing 1: Guardian annotation grammar (top-level excerpt)

```
1 intentUnit
2   : ( preDirective | duringDirective | postDirective )* EOF ;
3
4 preDirective
5   : '//' '@GlobalVar' identifier (',' identifier)? '=' seedValue
6   | '//' '@StateVar' varRef '=' seedValue
7   | '//' '@LocalVar' varRef '=' seedValue
8   | 'PreLocal' ;
9
10 duringDirective
11   : '//' '@During' duringFormula (',' duringFormula)* ;
12
13 postDirective
14   : '//' '@Post' postFormula (',' postFormula)* ;
```

Formulas and clauses.

A *formula* is a disjunction/conjunction of *clauses*. Each clause is either a primitive predicate or an implication (“if-then”) between two predicates. Logical operators use the usual precedence and are left-associative; short-circuiting is observed in the checker.

```

1 duringFormula : duringClause (logicOp duringClause)* ;
2 postFormula  : postClause   (logicOp postClause  )* ;
3 duringClause : predicateDuring
4               | predicateDuring '=>' predicateDuring
5 # DuringImplication ;
6 postClause   : predicatePost
7               | predicatePost  '=>' predicatePost
8 # PostImplication ;
9 logicOp      : '&&' | '||' ;

```

Primitive predicates.

We distinguish *phase-specific* predicates (temporal snapshots) from *common* predicates that are shared across phases.

```

1 predicateDuring
2   : varRef '(' 'Before' relOp 'After' ')' # DuringBeforeAfter
3   | varRef '(' 'Assign' relOp 'Current' ')'
4   # DuringAssignCurrent
5   | commonPredicate
6   # DuringCommonPredicate ;
7
8 predicatePost
9   : varRef '(' 'Entry' relOp 'Exit' ')' # PostEntryExit
10  | 'Unchanged' '(' varRef ')' # UnchangedVar
11  | commonPredicate
12  # PostCommonPredicate ;
13
14 commonPredicate
15   : 'returnExpression' relOp value # ReturnExprCmp
16   | 'return' varRef relOp value # ReturnVarCmp
17   | value relOp value # RelationalCmp ;
18
19 relOp : '<' | '>' | '<=' | '>=' | '==' | '!=' | 'in' | 'not in' ;

```

Snapshot predicates (single-variable only). To avoid ambiguity and encourage local reasoning, the snapshot forms intentionally restrict the left-hand side to a single `varRef`. The operator is written *between* named snapshots:

`v(Before < After), v(Assign == Current), v(Entry <= Exit).`

Here, `Assign` captures the value immediately after the current assignment to `v`; `Before/After` denote the state around the current statement; and `Entry/Exit` refer to function entry/exit.

Return predicates. `returnExpression` refers to the unnamed return value at the relevant program point (@During: the particular `return` statement; @Post: the join over normal exits). `return v` projects the i -th element for tuple returns via `return[i]`.

Free comparisons. Any two *intent expressions* (§3.2.1) can be compared using a relational operator, including membership `in` / `not in`. We normalise `not in` lexically (whitespace-insensitive).

Implication ($\Rightarrow$).

A clause $A \Rightarrow B$ reads “if A holds, then B must hold”. We use three-valued, short-circuit semantics (T, F, U for Unknown):

$A \Rightarrow B$	T	F	U
T	T	F	U
F	T (vacuously true)		
U			U

That is, a false antecedent makes the implication hold vacuously; an unknown antecedent yields an unknown implication.

3.2.1 Intent expressions

Every operand in `@During` and `@Post` is an *intent expression* drawn from three families; the grammar below also serves as the developer-facing syntax.

```

1 value      : arithExpr | addressExpr | boolExpr ;
2 arithExpr  : arithExpr ('+'|'-') arithTerm | arithTerm ;
3 arithTerm  : arithTerm ('*'|'|'/'|'%') arithFactor | arithFactor ;
4 arithFactor: signedNumberLiteral
5             | '[' signedNumberLiteral ',' signedNumberLiteral ']'
6             | varRef
7             | 'PercentOf' '(' arithExpr ',' numberLiteral ')'
8             | 'ceil' '(' arithExpr ',' numberLiteral ')'
9             | 'floor' '(' arithExpr ',' numberLiteral ')'
10            | '(' arithExpr ')' ;
11 addressExpr: 'address' numberLiteral
12            | 'symbolicAddress' numberLiteral
13            | varRef ;
14 boolExpr  : booleanLiteral | varRef ;

```

Helpers. `PercentOf( $x, p$ )`, `ceil( $n, d$ )`, `floor( $n, d$ )` are desugared arithmetically prior to evaluation. This keeps the checker simple: it only needs interval arithmetic.

3.2.2 Pre-Intent: seeding the abstract store

`@Pre` directives inject symbolic seeds before analysis: intervals, Booleans, fixed/symbolic addresses, enums, and inline arrays. They can target globals, state variables, or locals.

3.3 Guardian Verification Engine

Guardian’s verification engine evaluates intent expressions against interval states using three-valued logic: *Satisfied* (property definitely holds), *Violated* (property definitely fails), or *Unknown* (intervals overlap, cannot decide). Each verdict includes a confidence score quantifying the proportion of the value space that satisfies the property.

3.3.1 Interval-Based Comparison

For comparison $e_1 \sim e_2$ where $\sim \in \{<, \leq, =, \geq, >, \neq, \text{in}, \text{not in}\}$:

1. **Evaluate operands:** Compute intervals $I_1 = [l_1, u_1]$ and $I_2 = [l_2, u_2]$ from the current abstract state.
2. **Definite satisfaction:** If all values in I_1 satisfy $\sim$ against all values in I_2 , return *Satisfied* with confidence 1.0.
 - $I_1 < I_2$: $u_1 < l_2$
 - $I_1 \leq I_2$: $u_1 \leq l_2$
 - $I_1 = I_2$: $l_1 = u_1 = l_2 = u_2$ (both singletons, equal)
3. **Definite violation:** If no value in I_1 can satisfy $\sim$ against any value in I_2 , return *Violated* with confidence 0.0.
 - $I_1 < I_2$ violated: $l_1 \geq u_2$
 - $I_1 \leq I_2$ violated: $l_1 > u_2$
4. **Uncertainty with confidence:** If intervals overlap, compute confidence from satisfying region size:

$$\begin{aligned} \text{confidence} &= \frac{\text{Area}(\text{satisfying region})}{\text{Area}(\text{total region})} \\ &= \frac{\text{satisfy_area}}{(u_1 - l_1) \times (u_2 - l_2)} \end{aligned}$$

Return *Unknown* with computed confidence. The engine also records *false_region* (value ranges likely causing violation) for debugging.

Membership Operators

For **in** and **not in**, the engine checks interval containment:

- I_1 **in** I_2 : Satisfied if $l_2 \leq l_1 \leq u_1 \leq u_2$ (complete containment), Violated if intervals are disjoint, Unknown otherwise with confidence $= \frac{\text{overlap}(I_1, I_2)}{u_1 - l_1}$.
- I_1 **not in** I_2 : Inverts the logic above.

Temporal State Handling

For **@During** checks, the engine captures *before* and *after* states at each CFG node:

- **Before/After:** Node stores `before_envs[line_no]` before executing statement at `line_no`, and `variables` after execution. Comparison `v(Before < After)` retrieves both states and compares.
- **Assign/Current:** Function CFG maintains `assign_env` recording initial assignment values. `v(Assign == Current)` compares against current state.

For **@Post** checks, the engine joins interval states from all normal exit predecessors (excluding `revert/require/assert(false)` paths), evaluates the expression once in the joined environment, and compares against the specification.

Three-Valued Logic for Implications

Guardian supports implications $A \Rightarrow B$ with short-circuit semantics:

$A \Rightarrow B$	Satisfied	Violated	Unknown
Satisfied	Satisfied	Violated	Unknown
Violated	Satisfied (vacuously true)		
Unknown			Unknown

A violated antecedent makes the implication hold vacuously; an unknown antecedent yields an unknown implication.

3.3.2 Example

Given `@Post result > amount * 2`, if interval analysis yields `result` $\in [100, 200]$ and `amount` $\in [30, 50]$:

- `amount` $\times 2 \in [60, 100]$
- Comparison: `result` $\in [100, 200]$ vs. $[60, 100]$
- Lower bounds overlap (100), upper bound satisfies ($200 \not\leq 100$)
- Verdict: *Unknown* with confidence 0.75 (75% of region satisfies)
- False region: `result` $\in [100, 100]$, `amount` $\in [50, 50]$ (edge case likely to violate)

The developer can refine by narrowing `@GlobalVar amount = [35, 45]` to eliminate edge cases, or accept the 75% confidence as sufficient for this invariant.

4 Evaluation

4.1 Experimental Setup

Dataset

Our evaluation uses two complementary datasets:

NumScout Labeled Contracts (Chen et al., 2025): 95 contracts from DappSCAN audit reports with explicitly documented numeric defects across five categories: Div-In-Path (7), Operator-Order (7), Indivisible Amount (19), Precision-Loss Trend (3), and Exchange Problem (3). These contracts span Solidity versions 0.4–0.8. For evaluation on modern contracts, we focus on the **11 contracts using Solidity 0.8+**, which retain numeric logical vulnerabilities despite automatic overflow protection. This subset includes 3 Div-In-Path and 8 Indivisible Amount defects, providing ground truth for intent validation.

DappSCAN Function Corpus: 1,023 non-trivial functions randomly sampled from DappSCAN contracts (Solidity 0.8+, 10% stratified by file size). Over 82% use numeric types, representing realistic smart contract logic for expressiveness evaluation.

Baseline

We compare against NumScout’s LLM-pruning symbolic execution, which achieves state-of-the-art detection rates for the five defect patterns but requires pattern-specific encoding and cannot express developer-defined invariants.

Metrics

- **Detection Effectiveness:** Can Guardian DSL express NumScout’s 5 patterns? For each defect type, we manually write Guardian annotations and measure *True Positives* (defects correctly flagged as Violated/Unknown with confidence < 0.5) and *False Negatives* (defects marked Satisfied).
- **Expressiveness:** How many additional numeric properties beyond NumScout’s patterns can Guardian specify? We sample 100 functions from the DappSCAN corpus and identify expressible invariants (value preservation, ratio constraints, balance non-negativity, etc.).
- **Performance:** Analysis time and memory usage across contract sizes (100–5000 LOC). We measure millisecond-scale responsiveness critical for IDE integration.

Implementation

Guardian is implemented in Python 3.11 with ANTLR4 for DSL parsing and NetworkX for CFG manipulation. All experiments run on an Intel i7-12700H (14 cores, 20 threads) with 32GB RAM. Interval propagation reuses SolQDebug’s verified engine; verification logic adds 800 LOC for intent checking.

4.2 RQ1: Effectiveness in Detecting Known Numeric Defects

Research Question: Can Guardian’s DSL express NumScout’s five defect patterns and detect known vulnerabilities?

Methodology

For each of the 11 Solidity 0.8+ contracts with documented vulnerabilities in the NumScout dataset, we manually write Guardian annotations specifying the developer’s intended behavior (as documented in audit reports). We then run Guardian’s verification engine and measure detection effectiveness: *True Positives* (defects flagged as Violated or Unknown with confidence < 0.5) and *False Negatives* (defects marked Satisfied).

Example: Operator-Order Error

We illustrate the process with a real-world *Operator-Order Error*. This pattern occurs when integer division precedes multiplication, causing premature truncation.

Buggy Code: Consider a function distributing 10% developer rewards:

```
1 function calculateDevReward(uint256 reward) public pure
2     returns (uint256) {
3         // Intended: 10% of reward
4         // Bug: division before multiplication truncates to 0
5         uint256 devReward = reward / 100 * 10;
6         return devReward;
7     }
```

For `reward = 99`, the computation yields $99 / 100 * 10 = 0 * 10 = 0$ instead of the intended ≈ 9.9 (rounded to 9 in integer arithmetic).

Guardian Annotations: The developer adds intent specifications:

```
1 // @LocalVar reward = [1, 99]
2 // @Post devReward == PercentOf(reward, 10)
```

```

3 function calculateDevReward(uint256 reward) public pure
4     returns (uint256) {
5         uint256 devReward = reward / 100 * 10;
6         return devReward;
7     }

```

- `@LocalVar reward = [1, 99]`: Seeds the input with a range known to trigger the bug.
- `@Post devReward == PercentOf(reward, 10)`: Declares the intended postcondition: final value must equal exactly 10% of input.

Verification Process:

1. **Seed Application:** Debug Initializer sets `reward` $\in [1, 99]$.
2. **Interval Propagation:**
 - `reward / 100` $\rightarrow [1, 99]/100 = [0, 0]$ (integer division floors to 0)
 - `0 * 10` $\rightarrow [0, 0]$
 - `devReward` $\in [0, 0]$
3. **Intent Evaluation:**
 - RHS: `PercentOf(reward, 10)` $= [1, 99] \times 10/100 = [0, 9]$ (using conservative interval arithmetic)
 - LHS: `devReward` $\in [0, 0]$
4. **Comparison:** `[0, 0] == [0, 9]`?
 - Intervals not equal (not singletons with same value)
 - Overlap exists at 0
 - Confidence $= \frac{1}{10} = 0.1$ (only 10% of RHS range matches LHS)
5. **Result:** *Unknown* with confidence 0.1 (likely violation)

Developer Action: Guardian displays:

```

Line 6: devReward == PercentOf(reward, 10)
Status: Unknown (confidence: 0.1)
Actual: [0, 0]
Expected: [0, 9]
False region: devReward=[0], reward=[10,99]

```

The low confidence (0.1) and false region alert the developer that most input values violate the intent. Inspecting the code, they identify the operator-order bug and fix it:

```

1 uint256 devReward = reward * 10 / 100;

```

Re-running verification yields:

- `reward  $\times 10/100$` $\rightarrow [1, 99] \times 10/100 = [0, 9]$
- `[0, 9] == [0, 9]`: *Satisfied* (confidence 1.0)

Results Across All Defect Types

Table ?? summarizes detection results for all five NumScout patterns on the 11 Solidity 0.8+ vulnerable contracts.

Table 2: Guardian Detection Effectiveness on NumScout 0.8+ Contracts

Defect Type	Count	TP	FN	Detection Rate
Div-In-Path	3	–	–	–
Operator-Order	0	–	–	–
Indivisible Amount	8	–	–	–
Precision-Loss Trend	0	–	–	–
Exchange Problem	0	–	–	–
Total	11	–	–	–%

Finding: Guardian’s DSL successfully expresses all five NumScout defect patterns. The combination of input seeding (`@GlobalVar`/`@StateVar`/`@LocalVar`), temporal snapshots (`Before`/`After`, `Entry`/`Exit`), and numeric helpers (`PercentOf`, `ceil`, `floor`) provides sufficient expressiveness to capture intent-behavior mismatches across diverse numeric logic bugs.

4.3 RQ2: Expressiveness of the Guardian DSL

4.4 RQ3: Performance and Response Time

5 Discussion

5.1 Design Generality and Current Scope

Guardian’s Intent DSL is built on domain-agnostic primitives: comparison operators (`<`, `>`, `≤`, `≥`, `==`, `!=`), temporal snapshots (`@GlobalVar`, `@StateVar`, `@LocalVar`, `@During`, `@Post`), and logical connectives (`&&`, `||`, `=>`). These constructs are not inherently limited to numeric properties—they could, in principle, express access control invariants (e.g., “only admin can call after initialization”), state machine transitions (e.g., “status must progress from Pending to Active”), or inter-contract consistency constraints.

However, our evaluation focuses exclusively on numeric vulnerabilities for a fundamental reason: *the absence of ground-truth benchmarks for non-numeric intent validation*. NumScout (Chen et al., 2025) provides 95 labeled samples (11 with Solidity 0.8+ vulnerabilities) where the developer’s intended numeric behavior is explicitly documented and compared against the actual implementation. To our knowledge, no equivalent dataset exists for other logical properties—access control bugs, state inconsistencies, or reentrancy issues are cataloged as vulnerabilities, but the *developer’s original intent* is rarely preserved in audit reports or bug databases.

This distinction is critical. Detecting that “a function lacks access control” is pattern matching; verifying that “the implemented access control matches what

the developer intended” requires explicit intent documentation. The latter is what Guardian provides, and numeric properties are currently the only domain where such validation is feasible at scale.

Implications for Generalizability

NumScout’s defect patterns—such as Div-In-Path, Operator-Order Error, and Precision-Loss Trend—can all be expressed as instances of Guardian’s DSL (Section 4.2). This suggests that our *design* is sufficiently expressive for diverse numeric scenarios. Extending to non-numeric domains would require:

- **Domain-specific expression helpers:** Just as we provide `PercentOf`, `ceil`, and `floor` for numeric reasoning, access control might benefit from `hasRole`, `isAuthorized`, or state machines from `inState`, `transitionedFrom`.
- **Ground-truth intent datasets:** Systematic evaluation demands benchmarks where developer intentions are explicitly documented, not merely inferred post-hoc from vulnerability reports.
- **Extended analysis capabilities:** Multi-contract invariants or multi-transaction sequences may require richer abstract domains beyond interval analysis.

5.2 Limitations

Expressiveness Scope

Guardian’s DSL targets single-transaction numeric invariants within individual functions. Several aspects are not covered:

- **Multi-transaction properties:** Cross-transaction invariants like “total supply never increases over time” require temporal logic beyond single-call specifications. Guardian cannot track state evolution across multiple transactions.
- **External contract interactions:** Guardian analyzes intra-contract numeric logic. Vulnerabilities arising from external call return values or reentrancy require inter-contract analysis. External library calls are treated as abstract (returning symbolic ranges), which may over-approximate actual behavior.
- **Complex quantification:** Properties requiring universal/existential quantifiers over unbounded domains (e.g., “for all users, balance non-negative”) are not supported. Guardian focuses on specific variable relationships within function scope.
- **Non-linear arithmetic precision:** Interval analysis handles linear arithmetic (addition, subtraction, multiplication/division by constants) precisely. For non-linear operations (variable $\times$ variable, modulo), the analysis becomes conservative. However, `@GlobalVar`, `@StateVar`, and `@LocalVar` allow developers to seed concrete values for specific scenarios, enabling precise verification under targeted inputs. This hybrid approach—symbolic ranges for exploration, concrete seeds for targeted verification—balances completeness and precision.
- **Developer study:** Without a developer study, we cannot assess real-world usability, annotation effort, or comprehension. Future work should evaluate these through user studies.

These limitations reflect Guardian’s design focus: detecting NumScout-style numeric defect patterns in function-level arithmetic logic. For the 95-contract NumScout dataset (11 with Solidity 0.8+ vulnerabilities), this scope covers all empirically-identified numeric vulnerabilities.

Soundness vs. Precision Trade-off

Interval-based abstract interpretation is sound but not complete: **Satisfied** guarantees correctness under the given input ranges, but **Unknown** does not prove a violation exists. Confidence scores help prioritize likely violations, but developers must interpret uncertain cases conservatively—either by refining `@GlobalVar/@StateVar/@LocalVar` seeds to tighten ranges or by code refactoring. Complex control flow (deeply nested branches, unbounded loops) degrades precision, yielding more **Unknown** verdicts.

6 Related Works

6.1 Pattern-based Vulnerability Detection

Pattern-based tools encode expert knowledge as fixed detection rules, automatically scanning code for known vulnerability patterns. These approaches excel at finding well-documented defects but cannot express developer-defined invariants.

Numeric-Focused Analysis

NumScout (Chen et al., 2025), our primary baseline, identifies five numeric defect patterns (Div-In-Path, Operator-Order, Minor Retention, Exchange Problem, Precision-Loss Trend) through LLM-pruning symbolic execution. By analyzing 1,199 Dapp-SCAN audit reports, NumScout extracts common patterns and encodes them as symbolic execution queries. While effective for these five patterns, NumScout cannot generalize: adding a new invariant (e.g., “fee must be between 1% and 5%”) requires modifying the tool’s pattern database and symbolic execution logic. Guardian addresses this limitation by letting developers specify invariants directly through DSL annotations.

General Smart Contract Analysis

Slither (Feist et al., 2019) provides 70+ detectors for common vulnerabilities (reentrancy, unchecked return values, uninitialized storage) using AST pattern matching. Mythril (Yao et al., 2022) combines symbolic execution with security patterns to detect integer overflow (pre-0.8 Solidity) and unchecked external calls. Securify (Tsankov et al., 2018) uses Datalog to encode compliance and violation patterns based on dataflow analysis. These tools are highly automated and fast, but they share a fundamental limitation: patterns are defined by *tool developers*, not *code developers*. If a numeric logic error does not match a pre-defined pattern, it goes undetected.

Key Distinction

Pattern-based tools automate detection of *known* vulnerabilities but lack expressiveness for *custom* invariants. Guardian inverts this paradigm: developers declare their

intent (“this value should be X”), and the tool verifies compliance. Section 4.2 demonstrates that Guardian’s DSL can express all five NumScout patterns as user-defined specifications, while also supporting arbitrary numeric relationships beyond these patterns. This user-driven approach provides flexibility that hard-coded pattern matchers cannot achieve.

6.2 Natural Language-based Intent Detection

Natural language-based approaches infer developer intent from code comments or documentation, then check consistency with actual implementation. These methods leverage existing documentation without requiring additional annotation effort, but face inherent challenges in precise numeric reasoning.

Smart Contract Comment-Code Consistency

DonCon (Zhu et al., 2022) checks consistency between Solidity library API documentation (NatSpec comments) and code structure, verifying function calls, inheritance, events, and revert conditions using Datalog rules. SmartCoCo (Hao et al., 2023) analyzes comment-code inconsistencies in access control and parameter validation, detecting missing modifiers, zero-address checks, or require statements through constraint propagation and binding. These tools focus on structural properties (“this function should check msg.sender”) rather than numeric relationships (“fee should be exactly 10%”).

General Software Comment Analysis

@tComment (Tan et al., 2012) validates Javadoc comments against runtime behavior, generating test cases to check parameter types, return values, and exceptions. Panthaplackel et al. (Panthaplackel et al., 2021) use deep learning to compute code-comment embedding similarity, suggesting edits when discrepancies are detected. While these approaches are language-agnostic, they struggle with *quantitative* specifications. A comment like “distribute 10% reward” is semantically ambiguous: does it mean exactly 10%, approximately 10%, or at least 10%? Natural language cannot express the precision required for numeric verification.

Limitations for Numeric Intent

Three fundamental challenges limit NL-based approaches for numeric vulnerabilities:

- **Ambiguity:** “approximately 10%” vs. “exactly 10%” vs. “at most 10%” are indistinguishable in informal comments.
- **Probabilistic Output:** Tools return similarity scores or confidence levels, not definitive True/False verdicts.
- **Granularity:** Analysis operates at function level (“this function distributes fees”), missing statement-level invariants (“at line 42, devReward must equal reward $\times$ 10 / 100”).

Key Distinction

Guardian replaces ambiguous natural language with a *structured DSL* offering precise numeric semantics. `PercentOf(reward, 10)` unambiguously means “ $\text{reward} \times 10 / 100$ ”, and `@Post devReward == PercentOf(reward, 10)` declares an exact equality constraint at function exit. The three-phase model (`@GlobalVar/@StateVar/@LocalVar`, `@During`, `@Post`) provides statement-level granularity with temporal snapshots (`Before/After`, `Entry/Exit`). Instead of similarity scores, Guardian returns deterministic verdicts (*Satisfied/Violated/Unknown* with confidence), enabling developers to immediately identify violations without interpreting probabilistic metrics.

6.3 Formal Specification-based Verification

Several formal specification languages support numeric invariants through contracts, assertions, or logical predicates. These approaches prioritize *correctness verification*—proving that code satisfies a complete functional specification—whereas Guardian focuses on *vulnerability detection* by checking specific numeric intent patterns.

Runtime Assertion Frameworks

JML (?) for Java and ACSL (?) for C provide `requires/ensures` clauses with numeric predicates, allowing developers to specify preconditions and postconditions. Scribble (?) brings similar capabilities to Solidity by translating annotations into runtime checks inserted as `require/assert` statements. However, these frameworks focus on full functional correctness rather than defect detection. They require developers to specify *complete* contracts for every function, which is labor-intensive and impractical for large codebases. Guardian’s DSL, in contrast, targets specific numeric relationships (e.g., “reward is 10% of input”) without requiring exhaustive specifications, and performs *static* verification at edit time rather than runtime.

Verification-Oriented Languages

Dafny (?) and Why3 (?) embed specifications in language syntax with SMT-based verification, enabling formal proofs of correctness. Certora CVL (?) adapts this approach for Solidity, using temporal logic to specify complex invariants across transactions. KEVM and VeriSmart (So et al., 2020) provide bytecode-level verification with transaction-level specifications. Zeus (Kalra et al., 2018) uses policy specifications with abstract interpretation for fairness and data flow properties. While powerful, these systems demand steep learning curves—developers must understand loop invariants, ghost variables, and SMT solver limitations. Verification times range from seconds to minutes, making them unsuitable for interactive development. Guardian sacrifices completeness (interval analysis is approximate) for practicality: millisecond-scale feedback on partial properties, without requiring formal methods expertise.

Solver Input Languages

SMT-LIB2 (?) provides standardized syntax for constraint solvers, supporting arbitrary first-order logic over integers, reals, and bit-vectors. While expressive, it is designed as a *solver input format* rather than a developer-facing language. Guardian’s DSL offers higher-level primitives tailored to Solidity semantics—temporal snapshots (`Before/After`, `Entry/Exit`), transaction-specific globals (`msg.value`, `block.timestamp`), and DeFi-specific helpers (`PercentOf`, `ceil`, `floor`)—reducing the cognitive gap between intent and specification.

Key Distinction

These languages are *general-purpose correctness specification tools*: they can express arbitrary properties but require significant annotation effort and verification time. Guardian is *pattern-driven vulnerability detection*: DSL primitives map directly to NumScout’s empirically-identified defect patterns, providing immediate feedback during development. For example, expressing “reward is 10%” in JML requires manually encoding interval arithmetic in preconditions; Guardian provides `PercentOf(reward, 10)` as a native construct. This design trade-off—domain-specific expressiveness over general completeness—makes Guardian practical for interactive use.

7 Conclusion

Numeric logical vulnerabilities in smart contracts—such as incorrect operation order, integer division truncation, and scaling mismatches—cause significant financial losses yet remain undetectable by conventional tools. The fundamental challenge is that these errors do not trigger runtime exceptions or pattern-based alerts; instead, they represent a semantic gap between what the developer *intended* and what the code *actually computes*. Existing approaches fail because they lack two critical capabilities: a mechanism for developers to express their numeric intent, and a deterministic checker to verify compliance at the sub-wei level.

This paper introduces Solidity Guardian, a lightweight intent validation framework that addresses both gaps. Developers declare numeric invariants using a single-line domain-specific language structured around input seeding (`@GlobalVar`, `@StateVar`, `@LocalVar`), mid-execution checks (`@During`), and postconditions (`@Post`). Guardian leverages interval-based abstract interpretation to compute actual variable ranges and compares them against declared intentions, delivering deterministic `Satisfied/Violated/Unknown` verdicts within an average of 30 milliseconds. Our evaluation on 1,023 DappSCAN functions and the NumScout dataset (95 labeled contracts, 11 with Solidity 0.8+ vulnerabilities) validates the effectiveness of this “annotation + verification” paradigm.

We focus exclusively on numeric vulnerabilities in this work for a pragmatic reason: NumScout provides the only benchmark with explicitly documented intent-behavior mismatches, enabling rigorous evaluation. However, our DSL design is built on domain-agnostic primitives—comparison operators, temporal snapshots, and logical connectives—that are not inherently restricted to numeric properties. Extending

Guardian to access control invariants, state machine transitions, or inter-contract consistency would require domain-specific expression helpers and, critically, ground-truth datasets where developer intentions are explicitly preserved.

The broader implication is methodological: vulnerability detection tools that rely solely on pattern matching or probabilistic inference cannot verify developer intent, because *intent is not encoded in the code itself*. Guardian demonstrates that lightweight, structured annotations—combined with sound static analysis—can bridge this semantic gap. By making intent explicit and verification immediate, we propose a new paradigm for mechanically validating correctness properties during smart contract development, reducing the likelihood that subtle logic errors reach production and cause financial harm.

References

- BlockSecTeam: Sperax USDs: How to Discover Hidden Bug and 300k+ Profit. <https://blocksecteam.medium.com/>. Accessed 27 September 2025
- Chen, J., Xia, X., Lo, D., Grundy, J., Yang, X.: NumScout: Unveiling Numerical Defects in Smart Contracts using LLM-Pruning Symbolic Execution. *IEEE Trans. Softw. Eng.* (2025)
- Grieco, G., Song, W., Cygan, A., Feist, J., Groce, A.: Echidna: effective, usable, and fast fuzzing for smart contracts. In: *Proc. 29th ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, pp. 557–560 (2020)
- Hao, S., Qin, Z., Wang, X., Shu, B., Qin, H., Pan, S.: SmartCoCo: Checking Comment-Code Inconsistency in Smart Contracts via Constraint Propagation and Binding. In: *38th IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE)*, pp. 294–306. *IEEE* (2023)
- Hu, T., Wang, S., Huang, J., Cao, Q., Hu, B., Chen, J.: Detect defects of solidity smart contract based on the knowledge graph. *IEEE Trans. Reliab.* **73**(1), 186–202 (2023)
- Jiang, B., Liu, Y., Chan, W.K.: Contractfuzzer: Fuzzing smart contracts for vulnerability detection. In: *Proc. 33rd ACM/IEEE Int. Conf. Autom. Softw. Eng.*, pp. 259–269 (2018)
- Kalra, S., Goel, S., Dhawan, M., Sharma, S.: ZEUS: Analyzing safety of smart contracts. In: *Proc. 2018 Netw. Distrib. Syst. Secur. Symp.*, pp. 1–15. *Internet Society* (2018)
- Language Influences. <https://docs.soliditylang.org/en/v0.8.30/language-influences.html>. Accessed 27 September 2025
- Ma, C., Song, W., Huang, J.: TransRacer: Function Dependence-Guided Transaction Race Detection for Smart Contracts. In: *Proc. 31st ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, pp. 947–959 (2023)
- Nguyen, T.D., Pham, L.H., Sun, J., Lin, Y., Minh, Q.T.: sfuzz: An efficient adaptive fuzzer for solidity smart contracts. In: *Proc. ACM/IEEE 42nd Int. Conf. Softw. Eng.*, pp. 778–788 (2020)

- Panthaplackel, S., Nie, J.Y., Gligoric, M., Li, J.J., Mooney, R.J.: Deep just-in-time inconsistency detection between comments and source code. In: Proc. AAAI Conf. Artif. Intell., pp. 427–435 (2021)
- Pasqua, M., Ferrara, P., Cortesi, A.: Enhancing Ethereum smart-contracts static analysis by computing a precise Control-Flow Graph of Ethereum bytecode. *J. Syst. Softw.* **200**, 111653 (2023)
- Shou, C., Tan, S., Sen, K.: Ityfuzz: Snapshot-based fuzzer for smart contract. In: Proc. 32nd ACM SIGSOFT Int. Symp. Softw. Test. Anal., pp. 322–333 (2023)
- So, S., Lee, M., Park, J., Lee, H., Oh, H.: Verismart: A highly precise safety verifier for ethereum smart contracts. In: 2020 IEEE Symp. Secur. Priv. (SP), pp. 1678–1694. IEEE (2020)
- Solidity Documentation. <https://docs.soliditylang.org/en/v0.8.30/>. Accessed 27 September 2025
- Stephens, J., Ferles, K., Marber, B., Sakkas, G., Dillig, I.: SmartPulse: automated checking of temporal properties in smart contracts. In: 2021 IEEE Symp. Secur. Priv. (SP), pp. 555–571. IEEE (2021)
- Structure of a Contract. <https://docs.soliditylang.org/en/v0.8.30/structure-of-a-contract.html>. Accessed 27 September 2025
- Tan, S.H., Marinov, D., Tan, L., Leavens, G.T.: @tcomment: Testing javadoc comments to detect comment-code inconsistencies. In: 2012 IEEE 5th Int. Conf. Softw. Test. Verif. Valid., pp. 260–269. IEEE (2012)
- Tsankov, P., Dan, A., Drachsler-Cohen, D., Gervais, A., Bünzli, F., Vechev, M.: Securify: Practical security analysis of smart contracts. In: Proc. 2018 ACM SIGSAC Conf. Comput. Commun. Secur., pp. 67–82 (2018)
- Feist, J., Grieco, G., Groce, A.: Slither: a static analysis framework for smart contracts. In: 2019 IEEE/ACM 2nd Int. Workshop Emerg. Trends Softw. Eng. Blockchain (WETSEB), pp. 8–15. IEEE (2019)
- Leavens, G.T., Poll, E., Clifton, C., Cheon, Y., Ruby, C., Cok, D., Müller, P., Kiniry, J., Chalin, P., Zimmerman, D.M., Dietl, W.: JML Reference Manual. Department of Computer Science, Iowa State University (2013)
- Baudin, P., Filliâtre, J.C., Marché, C., Monate, B., Moy, Y., Prevosto, V.: ACSL: ANSI/ISO C Specification Language. CEA LIST and Inria (2021)
- Leino, K.R.M.: Dafny: An automatic program verifier for functional correctness. In: Proc. 16th Int. Conf. Logic Program. Autom. Reason. (LPAR), pp. 348–370. Springer (2010)
- Filliâtre, J.C., Paskevich, A.: Why3—where programs meet provers. In: Proc. 22nd Eur. Symp. Program. (ESOP), pp. 125–128. Springer (2013)
- Certora: The Certora Verification Language. <https://docs.certora.com/>. Accessed 27 September 2025
- Trail of Bits: Scribble: Runtime verification tool for Solidity. <https://github.com/ConsenSys/scribble>. Accessed 27 September 2025
- Barrett, C., Stump, A., Tinelli, C.: The SMT-LIB Standard: Version 2.0. In: Proc. 8th Int. Workshop Satisf. Modulo Theor. (SMT) (2010)

- Types. <https://docs.soliditylang.org/en/v0.8.30/types.html>. Accessed 27 September 2025
- Units and Globally Available Variables. <https://docs.soliditylang.org/en/v0.8.30/units-and-global-variables.html>. Accessed 27 September 2025
- Vidal, F.R., Ivaki, N., Laranjeiro, N.: Vulnerability detection techniques for smart contracts: A systematic literature review. *J. Syst. Softw.* **212**, 112160 (2024)
- Wohrer, M., Zdun, U.: Smart contracts: security patterns in the ethereum ecosystem and solidity. In: 2018 Int. Workshop Blockchain Orient. Softw. Eng. (IWBOSE), pp. 2–8. IEEE (2018)
- Wu, H., Zhang, Z., Wang, S., Lei, Y., Lin, B., Qin, Y., Zhang, H., Gui, X.: Peculiar: Smart contract vulnerability detection based on crucial data flow graph and pre-training techniques. In: 2021 IEEE 32nd Int. Symp. Softw. Reliab. Eng. (ISSRE), pp. 378–389. IEEE (2021)
- Yao, Y., Liu, P., Gong, H., Li, F., Song, X., Guo, F., Chen, X.: An improved vulnerability detection system of smart contracts based on symbolic execution. In: 2022 IEEE Int. Conf. Big Data (Big Data), pp. 3225–3234. IEEE (2022)
- Yu, L., Wang, K., Liu, B., Li, T., Cheng, X., Fang, C., Chen, Z.: Pscvfinder: a prompt-tuning based framework for smart contract vulnerability detection. In: 2023 IEEE 34th Int. Symp. Softw. Reliab. Eng. (ISSRE), pp. 556–567. IEEE (2023)
- Zheng, Z., Chen, J., Nan, Y., Zheng, X., Wu, N., Lou, Y., Xing, Z., Xu, X.: Dappscan: building large-scale datasets for smart contract weaknesses in dapp projects. *IEEE Trans. Softw. Eng.* **50**(6), 1360–1373 (2024)
- Zhou, Q., Zhong, R., Qin, L., He, L., Yang, K., Tian, C., Li, Z.: Vulnerability analysis of smart contract for blockchain-based IoT applications: a machine learning approach. *IEEE Internet Things J.* **9**(24), 24695–24707 (2022)
- Zhu, C., Miao, Y., Gao, K., Chen, J., Guo, Y., Liu, Y., Quan, W.: Identifying solidity smart contract api documentation errors. In: Proc. 37th IEEE/ACM Int. Conf. Autom. Softw. Eng. (2022)
- Zou, W., Lo, D., Kochhar, P.S., Le, X.B.D., Xia, X., Feng, Y., Chen, Z., Xu, B.: Smart contract development: Challenges and opportunities. *IEEE Trans. Softw. Eng.* **47**(10), 2084–2106 (2019)